
Q1. Explain function call including recursion.

Answer:

1. Function Call

A function call is the mechanism by which a program transfers control from the current function (the caller) to another function (the callee) to perform a specific task.

- **Mechanism:** When a function is called, the system pauses the execution of the current function and saves its context (local variables, return address) onto the **System Stack**.
- **Execution:** The control jumps to the first line of the called function.
- **Return:** Once the called function finishes, it returns a value (if any) and control pops back to the saved address in the caller function.

2. Recursion

Recursion is a programming technique where a function calls itself directly or indirectly to solve a problem. It breaks a complex problem into smaller, self-similar sub-problems.

Key Components of Recursion:

1. **Base Case:** The condition that stops the recursion. Without this, the function will call itself infinitely, leading to a "Stack Overflow" error.
2. **Recursive Step:** The part of the function where it calls itself with modified arguments to move closer to the base case.

How Recursion Works (The Stack):

Every recursive call creates a new "stack frame" in memory containing that specific call's local variables and parameters. These frames stack up until the base case is reached. Then, they resolve (pop) one by one from top to bottom.

Example Program: Factorial of a Number

Calculating $n!$ (n factorial) is a classic example. $5! = 5 \times 4 \times 3 \times 2 \times 1$.

C++

```
#include <iostream>
```

```
using namespace std;
```

```
// Recursive Function
```

```
int factorial(int n) {
```

```
    // 1. Base Case
```

```
    if (n <= 1) {
```

```
        return 1;
```

```
    }
```

```
    // 2. Recursive Step
```

```
    else {
```

```
        return n * factorial(n - 1);
```

```
    }
```

```
}
```

```
int main() {
```

```
    int num = 5;
```

```
    cout << "Factorial of " << num << " is: " << factorial(num);
```

```
    return 0;
```

```
}
```

Recursion vs. Iteration (Comparison):

- **Space:** Recursion uses more memory (stack overhead) than iteration (loops).
- **Readability:** Recursion is often cleaner and easier to write for problems like tree traversals or sorting (Merge Sort), whereas iteration is better for simple loops.

Q2. Write short notes on:

(a) Header File and Namespace

Header Files:

- **Definition:** Files ending with .h (or no extension in modern C++ like <iostream>) that contain declarations of functions, classes, and variables.
- **Purpose:** They allow code reusability. Instead of rewriting the code for cout or cin, we simply include the standard library header.
- **Syntax:** #include <filename> tells the preprocessor to paste the contents of the file into the program before compilation.
 - Standard headers: #include <iostream>
 - User-defined headers: #include "myheader.h"

Namespace:

- **Definition:** A namespace is a declarative region that provides a scope to the identifiers (names of types, functions, variables, etc.) inside it.
- **Purpose:** It prevents **Name Collisions**. For example, if you have a function print() and a library you use also has print(), the compiler gets confused. Namespaces solve this by grouping them (e.g., std::print vs myLib::print).
- **Usage:** using namespace std; brings all standard library names into the current scope so we don't have to type std::cout every time.

(b) Control Statements

Control statements dictate the flow of execution in a program. They are categorized into three types:

1. Selection Statements (Decision Making):

- **if, if-else:** Executes a block of code only if a condition is true.
- **switch:** Selects one of many code blocks to be executed based on a matching case value.

2. Iteration Statements (Loops):

- **for loop:** Used when the number of iterations is known beforehand.
- **while loop:** Used when the number of iterations is unknown; runs as long as the condition is true.
- **do-while loop:** Similar to while, but guarantees execution at least once.

3. Jump Statements:

- **break:** Exits the current loop or switch case immediately.
- **continue:** Skips the rest of the current loop iteration and starts the next one.
- **goto:** Jumps to a labeled part of the program (generally discouraged).

(c) Access Specifiers

Access specifiers determine the visibility and accessibility of class members (variables and functions). C++ has three types:

1. Private (Default):

- Members are accessible **only within the class** itself.
- They cannot be accessed by objects or derived classes.
- *Goal:* Data Hiding (Encapsulation).

2. Public:

- Members are accessible from **anywhere** inside or outside the program.
- Usually used for member functions (interfaces) that manipulate private data.

3. Protected:

- Members are accessible within the class and by **derived classes (child classes)**.
- They remain inaccessible to the outside world (like private).
- *Goal:* Inheritance.

Specifier	Inside Class	Inside Derived Class	Outside Class
Private	Yes	No	No
Protected	Yes	Yes	No
Public	Yes	Yes	Yes

Q3. Compare C and C++ and write down all features of C++.

Part A: Comparison (C vs C++)

Feature	C Language	C++ Language
Paradigm	Procedural Oriented Programming (POP).	Object-Oriented Programming (OOP) + Procedural.
Approach	Top-Down approach.	Bottom-Up approach.
Data Hiding	Not supported. Data is less secure.	Supported via Encapsulation (Private/Protected).
I/O Functions	Uses scanf() and printf().	Uses cin >> and cout << (Objects).
Overloading	Does not support function or operator overloading.	Supports both function and operator overloading.
Inheritance	No concept of inheritance.	Supports inheritance for reusability.
Memory	malloc() and free() functions.	new and delete operators.

Part B: Features of C++

1. **Object-Oriented:** C++ revolves around objects and classes, allowing implementation of real-world entities.
2. **Encapsulation:** Binding data and functions together into a single unit (class) and hiding sensitive data.
3. **Inheritance:** Capability of a class to derive properties and characteristics from another class. This promotes code reusability.
4. **Polymorphism:** The ability to take more than one form (e.g., Function Overloading and Virtual Functions).

5. **Abstraction:** Hiding internal details and showing only necessary functionality (e.g., using a TV remote without knowing the internal circuitry).
 6. **Rich Library (STL):** C++ provides the Standard Template Library (STL) containing algorithms, vectors, lists, and queues ready to use.
 7. **Speed:** Being a compiled language close to hardware, C++ is very fast, suitable for game engines and operating systems.
-

Q4. Explain Multilevel Inheritance and Copy Constructor with program.

1. Multilevel Inheritance

Definition:

In multilevel inheritance, a class is derived from another derived class. It forms a linear chain of inheritance.

- If Class B inherits from Class A, and Class C inherits from Class B, this is multilevel inheritance.
- **Flow:** Grandparent (A) $\rightarrow$ Parent (B) $\rightarrow$ Child (C).

Program Example:

C++

```
#include <iostream>
```

```
using namespace std;
```

```
class Grandparent {
```

```
public:
```

```
    void showGP() { cout << "I am the Grandparent." << endl; }
```

```
};
```

```
class Parent : public Grandparent {
```

```
public:
```

```
void showP() { cout << "I am the Parent." << endl; }  
};
```

```
class Child : public Parent {  
public:  
    void showC() { cout << "I am the Child." << endl; }  
};
```

```
int main() {  
    Child c;  
    c.showGP(); // Accessing Grandparent property  
    c.showP(); // Accessing Parent property  
    c.showC(); // Accessing Child property  
    return 0;  
}
```

2. Copy Constructor

Definition:

A copy constructor is a special constructor used to initialize a new object as a copy of an existing object of the same class.

- **Syntax:** ClassName(const ClassName &old_obj);
- It is called automatically when:
 1. An object is initialized with another object (A obj2 = obj1;).
 2. An object is passed by value to a function.

Program Example:

C++

```
#include <iostream>
```

```
using namespace std;

class Demo {
    int data;
public:
    // Parameterized Constructor
    Demo(int val) { data = val; }

    // Copy Constructor
    Demo(const Demo &obj) {
        data = obj.data;
        cout << "Copy Constructor Called!" << endl;
    }

    void display() { cout << "Value: " << data << endl; }
};

int main() {
    Demo obj1(100);    // Normal constructor
    Demo obj2 = obj1;  // Copy constructor called here

    obj1.display();
    obj2.display();
    return 0;
}
```

Q5. Write a program on Friend Function.

Theory:

A Friend Function is a function that is not a member of a class but is granted access to its private and protected members. It is declared inside the class with the keyword friend.

Key Characteristics:

- It is **not** called using an object (e.g., obj.func() is wrong). It is called like a normal function.
- It breaks encapsulation but is useful for specific cases like operator overloading or accessing data from two different classes.

Program Example:

Here is a program where a friend function accesses the private members of two different classes to find the sum.

C++

```
#include <iostream>
```

```
using namespace std;
```

```
// Forward declaration needed so ClassA knows ClassB exists
```

```
class ClassB;
```

```
class ClassA {
```

```
    int numA;
```

```
public:
```

```
    ClassA(int n) { numA = n; }
```

```
    // Declaring friend function
```

```
    friend void add(ClassA, ClassB);
```

```
};
```

```
class ClassB {  
    int numB;  
public:  
    ClassB(int n) { numB = n; }  
    // Declaring the SAME friend function  
    friend void add(ClassA, ClassB);  
};  
  
// Friend Function Definition  
// Note: It does not belong to ClassA or ClassB (no :: operator)  
void add(ClassA obj1, ClassB obj2) {  
    int sum = obj1.numA + obj2.numB; // Accessing private data directly  
    cout << "Sum of ClassA and ClassB values: " << sum << endl;  
}  
  
int main() {  
    ClassA a(10);  
    ClassB b(20);  
  
    // Calling friend function normally  
    add(a, b);  
  
    return 0;  
}
```